

Q1 What is inheritance? List and briefly explain its types?

→ Inheritance is an object-oriented programming concept where one class gets or reuses the properties and methods of another class. It helps you avoid repeating code and makes programs easier to manage.

Types of inheritance

- 1) Single inheritance: One child class inherits from one parent class. It is the simplest form, like a child learning from one parent.
- 2) Multiple Inheritance: One child class inherits from more than one parent class. Some languages support it directly, while others use alternatives like interfaces.
- 3) Multilevel inheritance: A class inherits from another class, and then a third class inherits from that second class. It looks like a chain.
- 4) Hierarchical inheritance: One parent class is inherited by many child classes. This is useful when several classes share the same base features.
- 5) Hybrid inheritance: A mix of two or more inheritance types is one structure. It combines different patterns to fit a complex design.

Q2

Q2 Define OOP. What are the Four pillars of OOP.

→ OOP stands for Object Oriented Programming. It is a way of writing code by organizing it around objects that contain both data and behavior, instead of focusing only on function and logic.

Four pillars of OOP

- i) Encapsulation: Keeping data and the method that work on it together in one unit, while controlling direct access to data.
- ii) Abstraction: Showing only the important details and hiding the complex implementation behind a simple interface.
- iii) Inheritance: Letting one class reuse the properties and method of another class, which reduces code duplication.
- iv) Polymorphism: Allowing the same action or method name to behave differently in different situation.

Q3 What is Polymorphism? List and explain its types.

→ Polymorphism means 'many forms'. In object-oriented programming, it lets the same interface, method, or operator behave differently depending on the object or data types used.

Types of Polymorphism

1) Compile-time polymorphism

Also called static polymorphism, this is decided when the code is compiled. It is commonly achieved through method overloading or operator overloading.

2) Run-time polymorphism

Also called dynamic polymorphism, this is decided when the program runs. It usually happen through method overriding and virtual functions in inheritance based system.

3) Parametric polymorphism

The same code works for many types without changing the logic, often through generics or templates.

Q4) Define Class. Define Object

→ Class

A class is a blueprint for creating object. It defines the properties and behaviors that the objects made from it will have.

In simple words, a class tell us.

- what data an object will store
- what actions an object can perform

For example, if we want to represent a student, the class may contain

- name
- roll number
- marks
- methods like display details.

A class by itself does not represent a real student. It is only a design or template.

Object

An object is an instance of class. It is created using the class and contain actual values.

If Student is a class, then Student₁, Student₂, and Student₃ can be objects of class.

Each object has.

- It own identity
- Its own state or values
- access to the methods defined in class